🐳 COMANDOS DOCKER E DOCKER COMPOSE - GUIA COMPLETO

🚀 COMANDOS BÁSICOS DO DOCKER

Gerenciamento de Containers

bash

```
# Executar container
docker run hello-world
docker run -it ubuntu /bin/bash          # Modo interativo
docker run -d nginx                      # Modo detached
docker run -p 8080:80 nginx              # Mapear portas
docker run -v /meu/path:/app nginx       # Mount volume
docker run --name meu-container nginx    # Nome personalizado

# Gerenciar containers
docker ps                                # Containers ativos
docker ps -a                             # Todos os containers
docker stop <container_id>               # Parar container
docker start <container_id>              # Iniciar container
docker restart <container_id>            # Reiniciar container
docker rm <container_id>                 # Remover container
docker rm -f <container_id>              # Forçar remoção
docker logs <container_id>               # Ver logs
docker logs -f <container_id>            # Seguir logs
```

```
docker exec -it <container_id> /bin/bash  # Executar comando no container
```

Gerenciamento de Imagens

bash

```
# Buscar imagens
docker search nginx
```

```bash
# Baixar imagens
docker pull nginx
docker pull nginx:latest
docker pull node:18-alpine

# Listar imagens
docker images
docker image ls

# Remover imagens
docker rmi <image_id>
docker rmi -f <image_id>                # Forçar remoção
docker image prune                      # Limpar imagens não usadas

# Informações do sistema
docker info
docker version
```

```bash
docker system df                        # Uso de disco
```

🏗️ COMANDOS AVANÇADOS DO DOCKER

Build de Imagens

```bash
# Build a partir do Dockerfile
docker build -t minha-imagem .
docker build -t minha-imagem:1.0 .
docker build -f Dockerfile.dev .        # Arquivo específico

# Build com argumentos
docker build --build-arg NODE_ENV=production -t minha-app .

# Multi-stage build
```

```bash
docker build -t minha-app:multi-stage .
```

Redes e Volumes

```bash
```

```
# Gerenciamento de redes
docker network ls                          # Listar redes
docker network create minha-rede           # Criar rede
docker network connect minha-rede <container>
docker network disconnect minha-rede <container>

# Gerenciamento de volumes
docker volume ls                           # Listar volumes
docker volume create meu-volume            # Criar volume
docker volume inspect meu-volume           # Inspecionar volume
docker volume rm meu-volume                # Remover volume
```

```
docker volume prune                        # Limpar volumes não usados
```

Inspeção e Monitoramento

bash

```
# Inspecionar recursos
docker inspect <container_id>
docker inspect <image_id>
docker stats                               # Estatísticas em tempo real
docker top <container_id>                  # Processos do container

# Health check
docker events                              # Eventos do Docker
```

```
docker wait <container_id>                 # Aguardar container parar
```

📝 DOCKERFILE - EXEMPLOS PRÁTICOS

Dockerfile para Aplicação Node.js

dockerfile

```
# Multi-stage build para Node.js
FROM node:18-alpine as builder

WORKDIR /app
COPY package*.json ./
```

```
RUN npm ci --only=production

FROM node:18-alpine

WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .

RUN addgroup -g 1001 -S nodejs
RUN adduser -S nextjs -u 1001

USER nextjs

EXPOSE 3000

      CMD ["npm", "start"]
```

Dockerfile para Aplicação Python

```
      dockerfile
```

```
FROM python:3.11-slim

WORKDIR /app

# Instalar dependências do sistema
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Instalar dependências Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código
COPY . .

EXPOSE 8000

      CMD ["python", "app.py"]
```

Dockerfile para Aplicação React

dockerfile

```
# Build stage
FROM node:18-alpine as build

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

🎯 DOCKER COMPOSE

Comandos Básicos

bash

```
# Executar serviços
docker-compose up
docker-compose up -d                    # Modo detached
docker-compose up --build               # Build e executa
docker-compose up service1 service2     # Serviços específicos

# Parar serviços
docker-compose down
docker-compose down -v                  # Remove volumes também
docker-compose stop                     # Para sem remover

# Gerenciamento
docker-compose ps                       # Status dos serviços
docker-compose logs                     # Logs de todos os serviços
docker-compose logs -f service1         # Seguir logs específicos
```

```
docker-compose exec service1 /bin/bash    # Executar comando
```

```
docker-compose restart service1           # Reiniciar serviço
```

Comandos Avançados

```
bash
```

```
# Build
docker-compose build
docker-compose build --no-cache          # Build sem cache

# Scale de serviços
docker-compose up --scale api=3 --scale worker=2

# Ambiente específico
docker-compose -f docker-compose.prod.yml up
docker-compose -f docker-compose.yml -f docker-compose.override.yml up

# Ver configuração
```

```
docker-compose config                    # Valida e mostra compose
```

📊 DOCKER COMPOSE - EXEMPLOS PRÁTICOS

docker-compose.yml para Aplicação Full-Stack

```
yaml
```

```
version: '3.8'

services:
  # Frontend React
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
```

```yaml
      - REACT_APP_API_URL=http://localhost:8000/api
    volumes:
      - ./frontend:/app
      - /app/node_modules
    depends_on:
      - backend
    networks:
      - app-network

  # Backend Node.js
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    volumes:
      - ./backend:/app
      - /app/node_modules
    depends_on:
      - db
    networks:
      - app-network

  # Banco de dados PostgreSQL
  db:
    image: postgres:15
    environment:
      - POSTGRES_DB=mydb
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app-network

  # Redis para cache
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
    networks:
      - app-network

volumes:
  postgres_data:
```

```
networks:
  app-network:
    driver: bridge
```

docker-compose.yml para Desenvolvimento

yaml

```
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile.dev
    ports:
      - "3000:3000"
    volumes:
      - .:/app
      - /app/node_modules
    environment:
      - CHOKIDAR_USEPOLLING=true
    command: npm start

  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: myapp
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

docker-compose.prod.yml para Produção

yaml

```yaml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/nginx/ssl
    depends_on:
      - app
    networks:
      - app-network

  app:
    build:
      context: .
      dockerfile: Dockerfile.prod
    environment:
      - NODE_ENV=production
      - DATABASE_URL=${DATABASE_URL}
    restart: unless-stopped
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

🔧 COMANDOS PARA DESENVOLVIMENTO

Desenvolvimento com Hot Reload

```bash
# Desenvolvimento com volumes para hot reload
docker-compose -f docker-compose.dev.yml up

# Rebuild específico
docker-compose build --no-cache frontend

# Executar comandos específicos
```

```
docker-compose exec backend npm test
```

```
docker-compose exec db psql -U user mydb
```

Debug e Troubleshooting

```
bash
```

```
# Ver variáveis de ambiente
docker-compose exec backend env

# Ver logs específicos
docker-compose logs --tail=100 backend

# Ver uso de recursos
docker-compose top

# Ver IP dos containers
```

```
docker-compose exec backend hostname -i
```

🚀 COMANDOS PARA PRODUÇÃO

Deploy em Produção

```
bash
```

```
# Build e deploy
docker-compose -f docker-compose.prod.yml up -d

# Atualizar serviços
docker-compose -f docker-compose.prod.yml pull
docker-compose -f docker-compose.prod.yml up -d

# Rollback
docker-compose -f docker-compose.prod.yml down
```

```
docker-compose -f docker-compose.prod.yml up -d --force-recreate
```

Monitoramento em Produção

bash

```
# Health check
docker-compose ps

# Backup de volumes
docker run --rm -v postgres_data:/source -v $(pwd):/backup alpine \
  tar -czf /backup/backup.tar.gz -C /source ./

# Limpeza
docker-compose down -v
```

docker system prune -a

🛠️ SCRIPTS ÚTEIS PARA DOCKER

Script de Desenvolvimento

bash

```
#!/bin/bash
# dev.sh

echo "Iniciando ambiente de desenvolvimento..."
```

docker-compose -f docker-compose.dev.yml up --build

Script de Deploy

bash

```
#!/bin/bash
# deploy.sh

echo "Fazendo build das imagens..."
```

```
docker-compose -f docker-compose.prod.yml build

echo "Subindo serviços..."
docker-compose -f docker-compose.prod.yml up -d

echo "Verificando saúde dos serviços..."
sleep 30

docker-compose -f docker-compose.prod.yml ps
```

Script de Backup

```
bash

#!/bin/bash
# backup.sh

BACKUP_DIR="./backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

echo "Criando backup em $BACKUP_DIR/backup_$TIMESTAMP.tar.gz"

docker-compose exec db pg_dump -U user mydb > dump.sql
tar -czf "$BACKUP_DIR/backup_$TIMESTAMP.tar.gz" dump.sql
rm dump.sql

echo "Backup concluído!"
```

💡 DICAS E BOAS PRÁTICAS

Otimização de Build

```
bash

# Use .dockerignore
echo "node_modules\nnpm-debug.log\n.git\n.env" > .dockerignore

# Build com cache inteligente

docker build --cache-from minha-imagem:latest -t minha-imagem:latest .
```

Segurança

bash

```bash
# Scan de vulnerabilidades
docker scan minha-imagem

# Executar como usuário não-root
docker run --user 1000:1000 minha-imagem

# Limitar recursos
docker run -m 512m --cpus=1.0 minha-imagem
```

Este guia cobre todos os comandos essenciais para trabalhar com Docker e Docker Compose em ambientes de desenvolvimento e produção! 🐳